

CLAUDE CERTIFICATION PROGRAM

Realistic Practice Exam Set

New scenarios for CCAO-F, CCDV-F, CCA-F, and CCAR-P

Exam	Format	Focus
Claude Certified Associate - Foundations (CCAO-F)	60 items 120 minutes \$99 720/1000 to pass	Professionals using Claude as a productivity tool (ops, marketing, PM, education, comms).
Claude Certified Developer - Foundations (CCDV-F)	53 items 120 minutes \$125 720/1000 to pass	Engineers who integrate Claude into applications and ship them (API, tools, MCP, agents, optimization, security).
Claude Certified Architect - Foundations (CCA-F)	60 scenario-based items 120 minutes \$125 720/1000 to pass	System designers making agent-architecture decisions and production-reliability tradeoffs.
Claude Certified Architect - Professional (CCAR-P)	63 items 120 minutes \$175 720/1000 to pass	Architects who own production Claude systems end-to-end. Recommended: 3+ yrs architecture, 6+ months production LLM work.

How to use this set

- Each exam section opens with its official blueprint (domains + weights) so you know where items concentrate. Question counts here are a sample per exam — the real exams are 53–63 items.
- Work each section under timed conditions with answers covered. Items are scenario-first and mirror the official style: one distractor usually "sounds responsible" but does not fix the real problem.
- The Answer Key & Rationale section (at the end) explains why the right answer is right AND why each trap is wrong — study the rationale, not just the letter.
- Passing is a scaled 720/1000 (criterion-referenced) on every exam. Note where you miss by domain and re-study that domain, matching the weighting.

Independent study material based on the official Anthropic Exam Guides (v1.0, effective July 2026). Not affiliated with or endorsed by Anthropic. These items are illustrative and are NOT from the live exam bank. All exams: scaled score 100-1000, passing cut score 720. This is Set 2 — different scenarios and question stems from Set 1, same blueprint weighting.

CCAO-F

Claude Certified Associate - Foundations

60 items | 120 minutes | \$99 | 720/1000 to pass

Blueprint: D1 Prompting & Task Execution 14% | D2 Output Evaluation & Validation 21% | D3 Product & Model Selection 12% | D4 Workflow Integration & Solution Design 16% | D5 Configuration & Knowledge Management 12% | D6 Governance, Risk & Responsible Use 15% | D7 Troubleshooting & Optimization 10%

Q1 *Single response*

D1 — PROMPTING AND TASK EXECUTION

A marketing associate asks Claude, "Write something good for our launch." The output is generic and off-brand. Without changing tools or model, which single change will most improve the result?

- A. Re-run the exact same prompt several times and pick the best of the batch.
- B. Give Claude the specifics: audience, channel, tone, product benefit, length, and a do/don't example, then iterate.
- C. Switch to the largest available model and keep the same vague prompt.
- D. Ask Claude to be 'more creative' and submit again.

Q2 *Single response*

D7 — TROUBLESHOOTING AND OPTIMIZATION

An HR associate builds a prompt that summarizes candidate feedback, but the summaries keep including the candidate's age and nationality — details the company forbids using in evaluations. The prompt does not ask for those fields. What is the best fix?

- A. Accept the summaries and manually delete the fields each time.
- B. Add an explicit instruction to exclude protected attributes (age, nationality, etc.) and to summarize only job-relevant criteria, then verify the output.
- C. Tell Claude to 'be fair' and resubmit.
- D. Stop using Claude for HR work entirely.

Q3 *Single response*

D3 — PRODUCT AND MODEL SELECTION

An analyst must reason through a complex, multi-step policy comparison across three long regulatory documents, where accuracy of the reasoning matters far more than speed or cost. Which choice best fits?

- A. Use a fast, low-cost model (Haiku) to keep it cheap.
- B. Use the most capable model (Opus), appropriate for complex multi-step reasoning where quality is the priority.
- C. Split the work across many tiny prompts on the cheapest model and staple the answers together.
- D. Turn off Projects to reduce context.

Q4 *Single response*

D4 — WORKFLOW INTEGRATION AND SOLUTION DESIGN

An operations lead wants to introduce Claude into a monthly reporting process that currently takes a team three days. Which approach best reflects sound workflow integration?

- A. Replace the entire team's judgment with Claude output published directly, with no review step.
- B. Map the current steps, identify where Claude adds value (drafting, data summarization), insert a human review/approval checkpoint, and define clear handoffs.
- C. Ask Claude to 'automate the whole report' in one prompt and ship whatever it returns.
- D. Keep the process identical and paste Claude outputs into a folder no one reads.

Q5 *Multiple response (select TWO)*

D6 — GOVERNANCE, RISK, AND RESPONSIBLE USE

Before rolling Claude out to a customer-facing content team, which TWO practices best reflect responsible use and governance?

- A. Define which use cases are appropriate/inappropriate and align them with the organization's AI policy.
- B. Establish a human review step for anything customer-facing or high-stakes before publication.

- C. Allow uploading of any customer data since the tool is internal.
- D. Skip disclosure and let Claude output be published verbatim to move faster.

CCDV-F

Claude Certified Developer - Foundations

53 items | 120 minutes | \$125 | 720/1000 to pass

Blueprint: D1 Agents & Workflows 14.7% | D2 Applications & Integration 33.1% | D3 Claude Code 3.1% | D4 Eval, Testing & Debugging 2.6% | D5 Model Selection & Optimization 16.8% | D6 Prompt & Context Engineering 11.0% | D7 Security & Safety 8.1% | D8 Tools & MCPs 10.6%

Q1 *Single response*

D2 — APPLICATIONS AND INTEGRATION (LARGEST DOMAIN, 33.1%)

You are building a live customer-facing chat feature. Responses can be long, and users complain the UI 'freezes' for several seconds before anything appears. Latency-to-first-token is the core complaint; total tokens are unchanged. What is the most appropriate change?

- A. Consume the Messages API in streaming mode so tokens render incrementally as they are generated.
- B. Switch the workload to the Message Batches API to reduce cost.
- C. Cap max_tokens at a very low value so responses finish sooner.
- D. Increase temperature so the model responds faster.

Q2 *Single response*

D4 — EVAL, TESTING, AND DEBUGGING

A tool-calling agent intermittently fails. The trace shows Claude returned a correct, well-formed tool_use request, but your application then threw an exception and never returned a tool_result to the model. Where is the bug, and what is the right fix?

- A. The model is hallucinating tools; switch to a bigger model.
- B. The bug is in your integration layer (tool execution / result handling); fix the executor and return a proper tool_result (or a structured error) to the model.
- C. The prompt is wrong; add 'do not fail' to the system prompt.
- D. Raise temperature so the agent avoids the failing tool.

Q3 *Single response*

D8 — TOOLS AND MCPs

You need Claude to reliably produce a company-standard incident report every time engineers ask, using a fixed template, tone, and section order — but it does NOT need to call any live external system. Which mechanism is the best fit?

- A. A custom Skill that packages the standard instructions, template, and formatting for reuse across sessions.
- B. An MCP server exposing a live database, since MCP is always the most reusable option.
- C. A built-in web-search tool.
- D. Hard-coding the template into a single throwaway prompt each time.

Q4 *Single response*

D1 — AGENTS AND WORKFLOWS

You are building an agent with the Claude Agent SDK. You want a deterministic guardrail that ALWAYS runs a safety/permission check before any tool that writes to production is executed, regardless of what the model decides. What is the correct mechanism?

- A. Add a sentence to the system prompt asking the model to check permissions first.
- B. Use a hook in the agent harness that deterministically intercepts tool calls and enforces the permission check before execution.
- C. Lower the temperature so the model is more careful.
- D. Rely on the model to remember the rule across turns.

Q5 *Single response*

D5 — MODEL SELECTION AND OPTIMIZATION

Your application pins its requests to a specific Claude model version. A new model version is released. What is the safest way to adopt it in production?

- A. Auto-upgrade all traffic immediately, since newer is always better.

- B.** Keep the version pinned, evaluate the new version against your eval set for quality/latency/cost and any breaking behavior changes, then roll it out gradually.
- C.** Never upgrade; pinning means you should stay on the launch version forever.
- D.** Switch based on which version has the shorter name.

Q6 *Single response*

D6 — PROMPT AND CONTEXT ENGINEERING

A long-running agent conversation degrades over time: it forgets earlier constraints, contradicts itself, and its context fills with verbose tool outputs. Which combination best addresses context drift and bloat?

- A.** Prune/compact tool outputs, keep durable constraints in a stable system prompt, and isolate subtasks via subagents so each works in a focused context.
- B.** Keep appending everything to one growing context and raise max_tokens.
- C.** Restart with an empty context and lose all task state.
- D.** Increase temperature so the model 'thinks differently.'

Q7 *Multiple response (select TWO)*

D7 — SECURITY AND SAFETY

Your Claude application uses API keys to reach Anthropic and several internal services. Which TWO practices best reflect secure secret and access management?

- A.** Store API keys in a secrets manager / environment configuration, never hard-coded in source or committed to version control.
- B.** Grant each component only the minimum scopes/permissions it needs (least privilege).
- C.** Embed the production key directly in the client-side app so it 'just works.'
- D.** Share one all-powerful key across every service to simplify rotation.

CCA-F

Claude Certified Architect - Foundations

60 scenario-based items | 120 minutes | \$125 | 720/1000 to pass

Blueprint: D1 Agentic Architecture & Orchestration 27% | D2 Claude Code Configuration & Workflows 20% | D3 Prompt Engineering & Structured Output 20% | D4 Tool Design & MCP Integration 18% | D5 Context Management & Reliability 15%

Scenario — Autonomous Code-Review Agent: You are architecting an internal agent that reviews pull requests. It runs in CI via Claude Code, reads diffs, checks them against team standards in CLAUDE.md, and can call MCP tools: `get_diff`, `run_tests`, `post_review_comment`, and `merge_pr`. The goal is to speed up reviews while never merging unsafe changes without human sign-off. The following questions relate to this system.

Q1 *Single response*

D2 — CLAUDE CODE CONFIGURATION & WORKFLOWS (20%)

The agent's coding standards, build commands, and review rules currently live in five different places, and reviewers disagree about which apply. Where should the authoritative, project-scoped instructions for the review agent live?

- A. In a well-structured CLAUDE.md at the repository, defining build/test commands, code-style guidelines, and project rules, respecting the CLAUDE.md hierarchy.
- B. Pasted fresh into every CI run by whoever triggers it.
- C. In each engineer's personal notes app.
- D. Only in the model's training data.

Q2 *Single response*

D4 — TOOL DESIGN & MCP INTEGRATION (18%)

merge_pr can irreversibly merge to main. The requirement is 'never merge unsafe changes without human sign-off.' Which design best satisfies this?

- A. Give the agent unrestricted merge_pr access so reviews finish fast.
- B. Gate merge_pr behind a required human approval (and/or passing-tests precondition) enforced by the harness, applying least privilege to the irreversible action.
- C. Rename merge_pr so the model rarely selects it.
- D. Let the agent merge, then notify a human afterward.

Q3 *Single response*

D1 — AGENTIC ARCHITECTURE & ORCHESTRATION (HEAVIEST, 27%)

On large PRs the single agent tries to hold the entire diff, all test output, and every standard in one context, and its review quality drops. Which orchestration change best preserves quality and maintainability?

- A. Decompose into a supervisor that spawns focused subagents (e.g., security review, style review, test-analysis), each with isolated context, then aggregate their findings.
- B. Feed the whole PR into one ever-larger context window and hope quality returns.
- C. Truncate the diff to the first file only.
- D. Remove the run_tests tool to shrink context.

Q4 *Single response*

D1 — AGENTIC ARCHITECTURE & ORCHESTRATION

Occasionally the agent loops: it calls run_tests, gets a failure, calls run_tests again unchanged, and repeats until it exhausts tokens. What is the most reliable production safeguard?

- A. Tell the agent in the prompt not to repeat itself.
- B. Enforce a hard iteration cap in the runner with a defined fallback (e.g., stop and request human review) when the limit is reached.
- C. Increase temperature so it varies its retries.
- D. Give it a second identical run_tests tool for variety.

Q5 *Single response*

D3 — PROMPT ENGINEERING & STRUCTURED OUTPUT (20%)

The CI system needs the agent's verdict as strict JSON: {"approve": true|false, "blocking_issues": ["..."], "needs_human": true|false}. Sometimes the agent adds commentary around the JSON, breaking the gate. Which is the most robust fix?

- A. Define the schema (structured output/tool), instruct the agent to emit only the JSON object, and validate on the CI side with a retry that returns the parse error to the model.

- B.** Grep the output for the word 'approve' and treat its presence as approval.
- C.** Ask the agent to 'try to output JSON' and accept whatever appears.
- D.** Delete everything before the first '{' and after the last '}' and trust the middle.

Q6 *Single response*

D5 — CONTEXT MANAGEMENT & RELIABILITY (15%)

Every CI run re-sends the same large CLAUDE.md standards and tool schemas as a static prefix ahead of the specific diff, and this is inflating per-run cost and latency. Which optimization fits best?

- A.** Mark the stable standards/schema prefix with `cache_control` so repeated runs reuse it from prompt cache at lower cost and latency.
- B.** Delete the standards from context and let the agent infer them.
- C.** Compress the standards to a single line to save tokens.
- D.** Run each PR twice to prime the cache.

Q7 *Single response*

D4 — TOOL DESIGN & MCP INTEGRATION

Your team also owns three other internal agents that all need the same `get_diff` and `run_tests` capabilities against the same backend. You want one implementation, maintained independently, that every agent can consume. What is the right architecture?

- A.** Copy the tool code into each of the four agents separately.
- B.** Expose `get_diff` and `run_tests` from a shared MCP server that all four agents connect to, maintained independently of any single agent.
- C.** Paste diffs and test logs into each agent's prompt manually.
- D.** Merge all four agents into one giant agent to avoid sharing.

CCAR-P

Claude Certified Architect - Professional

63 items | 120 minutes | \$175 | 720/1000 to pass

Blueprint: D1 Solution Design & Architecture 17% | D2 Models, Prompting & Context Engineering 13% | D3 Integration 19% (largest) | D4 Evaluation, Testing & Optimization 16% | D5 Governance, Safety & Risk 14% | D6 Stakeholder Communication & Lifecycle Management 14% | D7 Developer Productivity & Operational Enablement 7%

Context — Regulated Clinical-Documentation Assistant: A hospital network is deploying a Claude assistant that drafts clinical visit summaries from clinician notes and retrieves relevant guidelines via RAG. It must comply with health-data regulations, be defensible to auditors, and integrate with existing EHR systems. Several questions below draw on this context.

Q1 *Single response*

D5 — GOVERNANCE, SAFETY & RISK MANAGEMENT (14%)

The assistant will process protected health information under a regime like HIPAA. Which approach best reflects a defensible governance and safety design?

- A. Rely solely on the model's good behavior and a system-prompt instruction to 'protect patient privacy.'
- B. Design least-privilege access controls, data-handling and retention aligned to the regulation, human-in-the-loop for clinical content, audit logging, and documented guardrails/failure modes.
- C. Store all PHI in the prompt so the model always has full context, and log nothing to save space.
- D. Ship first and address compliance only if an auditor asks.

Q2 *Single response*

D3 — INTEGRATION (LARGEST, 19%)

The RAG assistant sometimes cites guidelines that are outdated or irrelevant to the patient's condition. Retrieval quality is the suspected issue. Which change most directly improves retrieval relevance?

- A. Match the chunking and indexing strategy to the guideline structure, ensure the index is refreshed as guidelines change, and tune retrieval (e.g., filters/metadata) to the query's clinical context.
- B. Increase the model size so it 'knows' the right guideline.
- C. Put all guidelines into the system prompt on every request and drop retrieval.
- D. Lower temperature to 0 and consider retrieval solved.

Q3 *Single response*

D6 — STAKEHOLDER COMMUNICATION & LIFECYCLE MANAGEMENT (14%)

During discovery, the chief medical officer says, 'Just make it write the final summaries automatically so clinicians don't have to.' You know clinician review is both a safety and a regulatory necessity. What is the most effective professional response?

- A. Agree and remove clinician review to meet the ask.
- B. Reframe the requirement: propose a human-in-the-loop design where Claude drafts and the clinician reviews/signs, explain the safety and compliance rationale and the residual risk of full automation, and align on that as the target.
- C. Refuse the meeting request and escalate over the CMO's head.
- D. Silently keep review in the design without telling the CMO, to avoid conflict.

Q4 *Single response*

D4 — EVALUATION, TESTING & OPTIMIZATION (16%)

You must demonstrate to a clinical governance board that the assistant is safe enough to pilot. Which evaluation strategy is most appropriate?

- A. Show a few impressive example outputs the team hand-picked.
- B. Build a representative, clinically-reviewed evaluation dataset with ground truth; measure accuracy, safety (e.g., harmful/incorrect guidance rate), latency, and cost; and define an explicit pilot go/no-go threshold with ongoing monitoring.
- C. Rely on user satisfaction surveys after launch as the only metric.
- D. Assume the model is safe because it is a capable general model.

Q5 *Single response*

D1 — SOLUTION DESIGN & ARCHITECTURE (17%)

Leadership wants one design that covers (1) high-volume, structured discharge-instruction generation with a fixed format, and (2) open-ended clinician Q&A over guidelines. Which architecture best serves both without over-engineering?

- A. Force both into a single free-roaming autonomous agent.
- B. Use a deterministic workflow for the fixed-format discharge-instruction path and an augmented-LLM/RAG pattern (retrieval + generation, with review) for the open-ended Q&A, selecting each pattern to its task.
- C. Use two identical maximum-capability agents and run them in parallel for everything.
- D. Handle everything in one enormous prompt with no retrieval or tools.

Q6 *Single response*

D2 — MODELS, PROMPTING & CONTEXT ENGINEERING (13%)

The assistant must apply the same large set of clinical formatting rules and safety guardrails on every request, and the team wants these reused consistently across multiple related assistants (discharge, referral, visit-summary) while controlling cost. Which strategy fits best?

- A. Rewrite the full rule set inline in each assistant's prompt, copy-pasted and independently edited.
- B. Modularize the shared rules (e.g., as reusable Skills/modular prompt components) and use prompt caching for the stable portions, so the rules are consistent and economical across assistants.
- C. Let each team invent its own rules to move faster.
- D. Put the rules only in a design doc and hope the model follows them.

Q7 *Single response*

D7 — DEVELOPER PRODUCTIVITY & OPERATIONAL ENABLEMENT (7%)

As the system moves to production, on-call engineers need to debug incidents quickly (e.g., a spike in low-confidence summaries) and the dev team needs consistent tooling. Which operational-enablement practice helps most?

- A. Give engineers no visibility and ask them to guess from user reports.
- B. Provide observability into requests/traces and standardized team tooling (including configured Claude Code workflows) so engineers can diagnose failures and reproduce issues consistently.
- C. Route every incident to the original architect personally.
- D. Disable logging in production to reduce overhead.

Answer Key & Rationale

Study the reasoning — the exams reward root-cause thinking and penalize "compensating-control theater."

CCAO-F Claude Certified Associate - Foundations

Q1 **Correct: B** (D1 — Prompting and Task Execution)

The failure is an under-specified prompt, not a model limitation. Providing concrete constraints (audience, channel, tone, benefit, length) plus a positive/negative example gives the model the task structure it needs, and iterating refines it. Re-rolling the same vague prompt (A, D) or upgrading the model (C) does not add the missing information.

Q2 **Correct: B** (D7 — Troubleshooting and Optimization)

Troubleshooting a poor output means adjusting the prompt to constrain it: explicitly exclude protected attributes and scope the summary to job-relevant criteria, then check the result. Manual deletion every time (A) is error-prone and doesn't fix the process; a vague 'be fair' (C) is not an actionable constraint; abandoning the tool (D) is unnecessary once the prompt is corrected.

Q3 **Correct: B** (D3 — Product and Model Selection)

Aligning model to task: complex, high-stakes multi-step reasoning is exactly where the top-tier model earns its cost/latency premium. A fast/cheap model (A) or fragmenting into tiny cheap prompts (C) risks reasoning errors on interdependent content; disabling Projects (D) removes helpful context and doesn't address model capability.

Q4 **Correct: B** (D4 — Workflow Integration and Solution Design)

Good integration analyzes the existing workflow, targets the steps where Claude genuinely helps, and builds in ownership, review, and handoffs. Publishing unreviewed output (A) ignores validation and accountability; a single 'automate everything' prompt (C) is neither reliable nor auditable; adding output nobody uses (D) delivers no value.

Q5 **Correct: A, B** (D6 — Governance, Risk, and Responsible Use)

Responsible rollout defines appropriate-use boundaries tied to policy (A) and enforces human review on customer-facing/high-stakes output (B). Uploading any customer data (C) ignores data-sensitivity rules, and publishing verbatim without review (D) removes the accountability control governance requires.

CCDV-F Claude Certified Developer - Foundations

Q1 **Correct: A** (D2 — Applications and Integration (largest domain, 33.1%))

The complaint is perceived latency (time to first token) in a live UI — streaming renders partial output immediately, eliminating the frozen-UI feel without changing total work. The Batches API (B) is for non-urgent async jobs and would make an interactive feature worse. Capping max_tokens (C) truncates answers; temperature (D) has no effect on speed.

Q2 **Correct: B** (D4 — Eval, Testing, and Debugging)

Trace analysis isolates origin: the model's tool_use was correct, and the exception occurred in your code executing the tool and returning the result. That's an integration-layer bug — fix the executor and always return a tool_result or a structured error the model can react to. A bigger model (A), prompt pleas (C), or temperature (D) don't touch the actual defect in your code.

Q3 **Correct: A** (D8 — Tools and MCPs)

When the need is packaged, reusable instructions/formatting with no live system access, a Skill is the right abstraction. An MCP server (B) is for connecting to external tools/data the task doesn't require here; web search (C) is unrelated; a throwaway prompt (D) is neither reusable nor consistent. This is the built-in-tool vs. custom-tool vs. Skill vs. MCP tradeoff the domain tests.

Q4 **Correct: B** (D1 — Agents and Workflows)

Deterministic, always-on enforcement belongs in a hook in the harness that intercepts tool invocations before execution — it does not depend on the model's cooperation. A prompt instruction (A) or relying on memory (D) is best-effort, not guaranteed; temperature (C) doesn't enforce anything. Hooks are the SDK mechanism for deterministic actions/guardrails.

Q5 Correct: B (D5 — Model Selection and Optimization)

Model releases can change behavior, so responsible adoption keeps the pin, runs the new version through your evaluation set (quality, latency, cost, breaking changes), and rolls out gradually. Blind auto-upgrade (A) risks silent regressions; never upgrading (C) forgoes real improvements and eventual deprecation; (D) is arbitrary. Version pinning exists precisely to make upgrades a deliberate, tested step.

Q6 Correct: A (D6 — Prompt and Context Engineering)

Context engineering for long sessions prunes/compacts noisy tool output, anchors durable rules in a stable system prompt, and isolates subtasks through subagents to prevent bloat and drift. Endlessly appending (B) worsens both; a full reset (C) loses needed state; temperature (D) is irrelevant to context management.

Q7 Correct: A, B (D7 — Security and Safety)

Secrets belong in a secrets manager/env config out of source control (A), and each component should hold only the minimum access it needs (B). Embedding a production key client-side (C) exposes it to anyone; a single omnipotent shared key (D) maximizes blast radius if leaked. Both wrong options increase risk rather than reduce it.

CCA-F Claude Certified Architect - Foundations

Q1 Correct: A (D2 — Claude Code Configuration & Workflows (20%))

CLAUDE.md is the project instruction manual: a structured, repo-scoped file for build/test commands, style guidelines, and rules, with a defined user/workspace hierarchy. Pasting per-run (B) or personal notes (C) are inconsistent and unauthoritative; the model does not carry your private standards from training (D).

Q2 Correct: B (D4 — Tool Design & MCP Integration (18%))

An irreversible, high-impact action needs an enforced approval gate (human sign-off and/or green tests) applied by the harness — least privilege. Unrestricted merge (A) violates the safety requirement; obscuring the name (C) is security-by-obscurity; merge-then-notify (D) is exactly the unsigned-off merge the requirement forbids.

Q3 Correct: A (D1 — Agentic Architecture & Orchestration (heaviest, 27%))

A supervisor/subagent (hub-and-spoke) design isolates each concern's context, preserving quality on large inputs while staying maintainable. Merely enlarging the window (B) still crams unrelated context together; truncating the diff (C) skips real changes; removing run_tests (D) breaks needed functionality.

Q4 Correct: B (D1 — Agentic Architecture & Orchestration)

Runaway loops are bounded deterministically in the harness: a hard iteration cap plus a safe fallback guarantees termination and escalation. A prompt request (A) isn't enforceable; temperature (C) doesn't bound iterations; duplicating the tool (D) adds confusion, not safety.

Q5 Correct: A (D3 — Prompt Engineering & Structured Output (20%))

Robust structured output = explicit schema + emit-only-payload instruction + validation-with-retry feeding the error back. Grepping for a keyword (B) is dangerously imprecise (it could match 'do not approve'); a soft request (C) still varies; naive bracket-stripping (D) breaks on nested or malformed content.

Q6 Correct: A (D5 — Context Management & Reliability (15%))

A large stable prefix reused across runs is the textbook prompt-caching case: cache it to cut cost and latency without changing behavior. Removing (B) or over-compressing (C) the standards degrades review quality; running twice (D) increases cost rather than reducing it.

Q7 Correct: B (D4 — Tool Design & MCP Integration)

A shared capability needed by multiple agents and maintained independently is the canonical MCP server case. Copying code four times (A) multiplies maintenance and drift; manual pasting (C) is neither live nor scalable; collapsing four agents into one (D) sacrifices separation of concerns to dodge a problem MCP already solves.

CCAR-P Claude Certified Architect - Professional

Q1 Correct: B (D5 — Governance, Safety & Risk Management (14%))

A defensible regulated design combines least-privilege access, regulation-aligned data handling/retention, human-in-the-loop on clinical output, audit logging, and documented guardrails and failure modes. A prompt-level plea (A) is not a control; maximizing PHI exposure with no logging (C) violates both privacy and auditability; deferring compliance (D) is indefensible in a regulated setting.

Q2 Correct: A (D3 — Integration (largest, 19%))

Retrieval relevance is an integration/pipeline problem: appropriate chunking + indexing, index freshness, and context-aware retrieval (metadata/filters) target the actual cause. A bigger model (B) doesn't fix retrieval; stuffing every guideline into the prompt (C) is costly, hits context limits, and abandons scalable retrieval; temperature (D) is unrelated to which documents are retrieved.

Q3 Correct: B (D6 — Stakeholder Communication & Lifecycle Management (14%))

The professional skill is structured discovery plus trade-off communication: convert an unsafe ask into a compliant human-in-the-loop design and explain the safety/regulatory rationale and residual risk so the sponsor can decide with full information. Complying blindly (A) creates safety and legal exposure; going over their head (C) is needless conflict; hiding the decision (D) undermines the transparency an accountable sponsor needs.

Q4 Correct: B (D4 — Evaluation, Testing & Optimization (16%))

A board-defensible strategy uses a representative, expert-reviewed dataset with ground truth and multi-dimensional metrics (accuracy, safety, latency, cost), plus an explicit threshold and ongoing monitoring. Cherry-picked demos (A), post-hoc surveys alone (C), or assuming safety from general capability (D) are precisely the anti-patterns a governance board should reject.

Q5 Correct: B (D1 — Solution Design & Architecture (17%))

Match the pattern to the task: a fixed-format, high-volume job fits a deterministic workflow (predictable, auditable), while open-ended guideline Q&A fits an augmented-LLM/RAG pattern with review. Forcing both into one autonomous agent (A) or duplicated max-capability agents (C) adds variance and cost with no benefit; a single tool-less mega-prompt (D) can neither retrieve current guidelines nor scale reliably.

Q6 Correct: B (D2 — Models, Prompting & Context Engineering (13%))

Prompt-reuse strategy: factor shared rules into reusable modular components/Skills and cache the stable parts, giving consistency across related assistants at lower cost. Copy-pasting inline (A) causes drift and duplicate maintenance; per-team improvisation (C) destroys consistency and safety; a design doc the model never sees (D) enforces nothing at runtime.

Q7 Correct: B (D7 — Developer Productivity & Operational Enablement (7%))

Operational enablement means giving the team observability (traces/metrics) and standardized, configured tooling so incidents can be diagnosed and reproduced consistently. Guessing from user reports (A) or funneling everything to one person (C) doesn't scale; disabling logging (D) removes exactly the signal on-call needs to debug.

Cross-Exam Themes to Master

Streaming vs. batch — know the complaint

A frozen live UI is a time-to-first-token problem → streaming mode renders partial output immediately. The Batches API is the opposite tool (non-urgent async, 24h, cheaper). Capping max_tokens or changing temperature does not fix perceived latency.

Hooks = deterministic enforcement

When a check must ALWAYS run before a sensitive/production tool regardless of the model, put it in a hook in the harness that intercepts tool calls. System-prompt instructions and "remember the rule" are best-effort, not guaranteed.

Least privilege for irreversible tools

merge_pr, process_refund, account-deletion, anything irreversible → gate behind enforced human sign-off and/or a passing-tests precondition. Merge-then-notify and renaming the tool are not controls.

Isolate origin from the trace

A correct tool_use followed by an app exception (no tool_result returned) is an integration-layer bug — fix your executor and always return a tool_result or structured error. It is not a model, prompt, or temperature problem.

Version pinning is deliberate upgrade

New model version → keep the pin, run it through your eval set (quality/latency/cost/breaking changes), roll out gradually. Neither auto-upgrade-everything nor never-upgrade is correct.

CLAUDE.md is the source of truth

Authoritative, repo-scoped build/test commands, style, and rules belong in a structured CLAUDE.md (with its user/workspace hierarchy) — not pasted per-run, personal notes, or assumed from training.

Skill vs. MCP vs. built-in vs. custom tool

Reusable packaged instructions/format with no live system → Skill. Shared capability multiple apps need, maintained independently → MCP server. Match the abstraction to the need; do not reach for MCP reflexively.

Context drift & bloat

Long sessions: prune/compact tool output, anchor durable constraints in a stable system prompt, isolate subtasks via subagents. Endless appending or a full reset are both wrong.

Structured output discipline

Schema in system/tool definition + emit-only-payload + validation-with-retry feeding the error back. Keyword-grep ("approve"), soft requests, and naive bracket-stripping all fail on real output variation.

Secrets & least privilege

API keys in a secrets manager / env config, never in source or client-side. Scope each component to the minimum it needs. One omnipotent shared key maximizes blast radius.

RAG relevance is a pipeline problem

Outdated/irrelevant citations → fix chunking, indexing, index freshness, and context-aware retrieval (metadata/filters). A bigger model or lower temperature does not fix which documents are retrieved.

Stakeholders + evaluation (Professional)

Convert unsafe/absolute asks ("fully automate clinical summaries") into a human-in-the-loop design with explained residual risk and an agreed target. Defensible evaluation = representative, expert-reviewed dataset + multi-metric + explicit go/no-go threshold + monitoring — never cherry-picked demos.